Shortest Path Algorithms

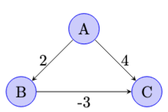
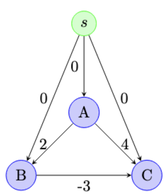
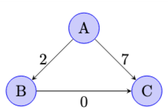
Aidan Johnson, Hunter Pearson, Sam Harris

University of Colorado Denver

Johnson's Algorithm

- Used to find shortest path distance between any pair of vertices
- Works for graphs with negative edge weights, but not negative cycles
- Combines Bellman-Ford and Dijkstra's algorithms
- Time Complexity: $O(|V|^2 \log |V| + |V||E|)$
  - $O(|V||E|)$ for Bellman-Ford stage
  - $|V|$ iterations of Dijkstra's algorithm (Fibonacci implementation) which is $O(|V| \log |V| + |E|)$
- Faster than Floyd-Warshall ($O(|V|^3)$) on sparse graphs.

Johnson's Algorithm Example

Step #	Description	Graph/Calculations
0	Original weighted directed graph	
1	Add auxiliary source node s with zero-weight edges	
2	Run Bellman-Ford from s to compute $h(v)$, the shortest path distance from s to v	• $h(A) = 0$ • $h(B) = 0$ • $h(C) = -3$
3	Re-weight edges to removed negative weights, using the following equation: $w'(u,v) = w(u,v) + h(u) - h(v)$	 • $w'(A,B) = 2 + 0 - 0 = 2$ • $w'(A,C) = 4 + 0 - (-3) = 7$ • $w'(B,C) = -3 + 0 - (-3) = 0$
4	Run Dijkstra from each node on reweighted graph, to get all shortest past pairwise distances $d'(u,v)$ in the transformed graph(using Fibonacci heap implementation)	$\begin{array}{c\|ccc} & A & B & C \\ \hline A & 0 & 2 & 2 \\ B & \infty & 0 & 0 \\ C & \infty & \infty & 0 \end{array}$ $d'(u,v)$
5	Convert distances in transformed graph to corresponding distances in original graph using the following equation: $d(u,v) = d'(u,v) - h(u) + h(v)$	$\begin{array}{c\|ccc} & A & B & C \\ \hline A & 0 & 2 & -1 \\ B & \infty & 0 & -3 \\ C & \infty & \infty & 0 \end{array}$ $d(u,v)$

Dial's Implementation

Dial's implementation of Dijkstra's algorithm uses a bucket data structure to significantly reduce the time taken for node selection. Given a graph $G = (V, E)$ with positive integer edge weights and maximum edge weight M, we create $M + 1$ buckets. These buckets are then used to denote the distance of each node compared to smallest distance temporary node. This method relies on the fact that for each iteration of Dijkstra's algorithm, the distance will become no smaller than the current smallest distance and no larger than the current smallest distance plus M.

This new data structure allows adding, deleting, or selecting elements to run in $O(1)$ time. Because of this, the time complexity relies only on searching for empty buckets and updating distances. Searching for empty buckets will loop through the M buckets at most $|V| - 1$ times and updating distances occurs at most $|E|$ times. Thus the time complexity of this implementation is $O(|E| + M|V|)$.

Theoretically, $O(|E| + M|V|)$ is not better than the traditional implementation. This is because it is only psuedopolynomial, relying on the size of the edge capacities. In applications, the edge weights are often much smaller than the number of nodes. This makes this algorithm much faster than traditional implementations in practice.

Dial's Implementation:

Start with s in the $0 \mod (M+1)$ bucket
For any edge $(s, i) \in E$, set $d(i) = c(s, i)$.
Move sequentially through the buckets
If a bucket is non-empty
  Remove the first node i
  For any $(i, j) \in E$
    If $d(i) + c(i, j) < d(j)$
      Remove j from its current bucket
      Place j in bucket:
        $d(i) + c(i, j) \mod M + 1$
      Set prev$(j) = i$
Stop when no nodes in buckets

Shortest Path Problem

Given a graph $G = (V, E)$ and weights $w : E \to \mathbb{R}$, find the shortest path from a node to another node. Extensions of this problem include finding the shortest paths from a given node to all other nodes (e.g. Dijkstra's Algorithm, Dial's Implementation, and the Bellman-Ford Algorithm) and finding the shortest paths from every node to every other node (Johnson's Algorithm).

Bellman-Ford Example

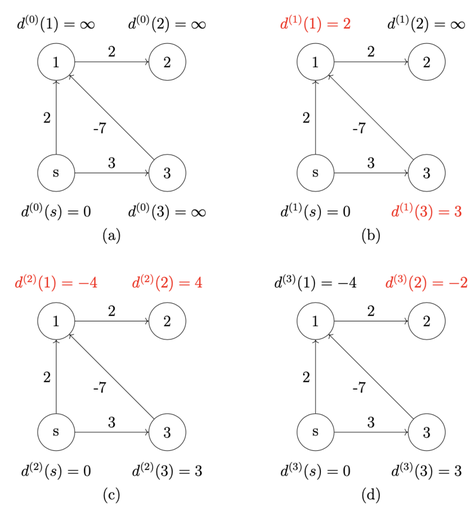
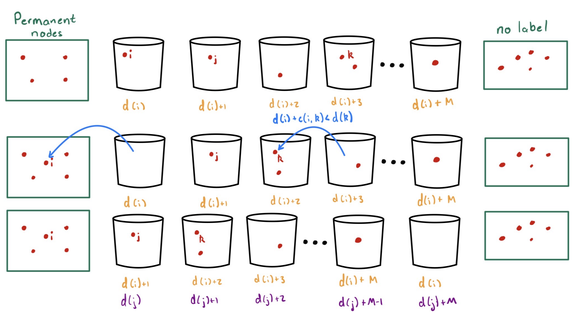


Figure 1: Network (a) is the original network with the initial distance labels of 0 and ∞. Network (b) is after the first iteration. Note that only the labels on 1 and 3 changed since 2 can not be reached by a path of length 1. Network (c) is after the second iteration. Note that the label on 1 changed again because a cheaper path using 2 edges was found and the label on 2 was also updated. Network (d) is after the third and final iteration. The distance label on 2 changed because a cheaper path using 3 edges was found.

Dial's Example



The Bellman-Ford Algorithm

The Bellman-Ford algorithm finds all the shortest paths from one vertex, the source, to all other vertices in a network. This algorithm utilizes dynamic programming and looks at every edge during each iteration which in turn implies that it can be used on networks with negative edge weights. Additionally, it can be used to detect negative cycles.

Let s be the source node and let $d^{(k)}(v)$ be the cost of the shortest path from s to v with at most k edges in it. Set the distance label on s to be 0 and the distance labels on the $|V| - 1$ other vertices to be infinity. The algorithm will need to perform $|V| - 1$ iterations since a path between two vertices can have at most $|V| - 1$ edges. During iteration k, the algorithm updates each $v \in V$ by comparing the current shortest path on $k - 1$ edges to the cost of reaching v through any incoming edges (u, v) using $d^{(k-1)}(u) + w_{uv}$. If we perform one more iteration (for a total of $|V|$) and a distance label changes, a negative cycle must be present.

During each of the $|V| - 1$ iterations each edge is examined which shows that Bellman-Ford has a run time of $O(|V| \cdot |E|)$. Note that this is slower than Dijkstra's; however, Bellman-Ford is considered to be easier to implement and has the added plus of being able to handle negative edge weights and detect negative cycles.

Bellman-Ford Algorithm:

$d^{(0)}(s) = 0$
$d^{(0)}(v) = \infty \quad \forall v \in V \setminus \{s\}$
For $k = 1, \ldots, |V| - 1$:
  For all $(u, v) \in E$:
    $d^{(k)}(v) = \min\{d^{(k-1)}(v),\ d^{(k-1)}(u) + w_{uv}\}$

References

[1] Stanford University CS161 Course Staff.
Lecture 12: Dynamic programming — bellman-ford and floyd-warshall.
https://stanford-cs161.github.io/winter2022/assets/files/lecture12-notes.pdf, 2022.

[2] Bernhard Korte and Jens Vygen.
Combinatorial Optimization: Theory and Algorithms.
Springer, 6 edition, 2018.

[3] MIT OpenCourseWare.
6.006 introduction to algorithms, lecture 14.
https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/, 2020.

[4] Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin.
Network Flows: Theory, Algorithms, and Applications.
Prentice-Hall, 1993.